МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Кафедра «ИТ»

Лабораторная работа № 4
по дисциплине «Введение в информационные технологии»
по теме: «ОРГАНИЗАЦИЯ PYTHON-ПРОЕКТА»

Выполнил студент группы:
БВТ2604
Оборотов Н.А.
Проверил:
Павликов А.Е.
Мосева М.С.

Москва 2026

Оглавление

Введение..... 3
Выполнение 4
Вывод..... 14

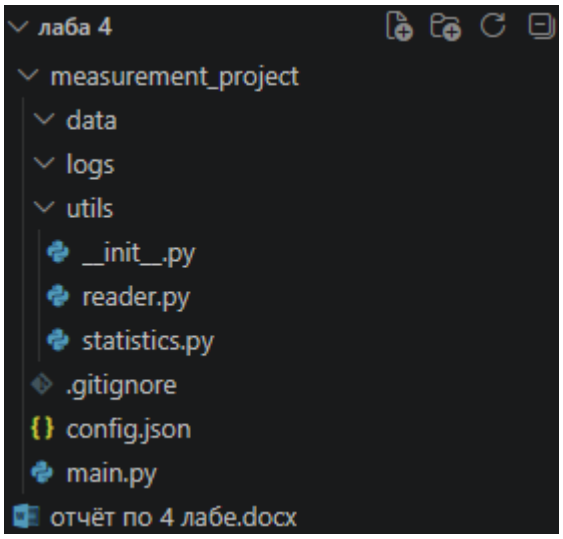
Цель и задачи работы:

Целью данной лабораторной работы является: изучить принципы организации Python-проекта, научиться разделять программу на модули и пакеты, использовать различные способы импорта, работать с файловой системой и операционной системой, обрабатывать исключения, применять логирование, использовать конфигурационные файлы, создавать виртуальные окружения и управлять зависимостями проекта.

Задачи: В ходе выполнения лабораторной работы необходимо научиться организовывать структуру проекта, распределять программный код между несколькими файлами, корректно обрабатывать ошибки, сохранять диагностическую информацию в журнал, выносить изменяемые параметры в конфигурационный файл и создавать воспроизводимое окружение проекта.

Выполнение

Задание 1



создание структуры проекта

- main и utils – код
- data – данные
- config.json – настройки
- .gitignore logs – служебное

Задание 2

```python
def calculate_average(numbers):#среднее арифметическое
    sum=0
    for chisla in numbers:
        sum+=chisla
    return sum/len(numbers)



def find_min(numbers):#минимальое
    mini=numbers[0]
    for chisla in numbers:
        if chisla<mini:
            mini=chisla
    return mini


def find_max(numbers):
    maxi=numbers[0]
    for chisla in numbers:
        if chisla>maxi:
            maxi=chisla
    return maxi
```

создание функций ср. ар. ,минимального, максимального

```python
from utils.statistics import calculate_average,find_min,find_max
test=[4,34,532,325,654,-546,1134,452,123,75,2134,12]
print(calculate_average(test))
print(find_min(test))
print(find_max(test))
```

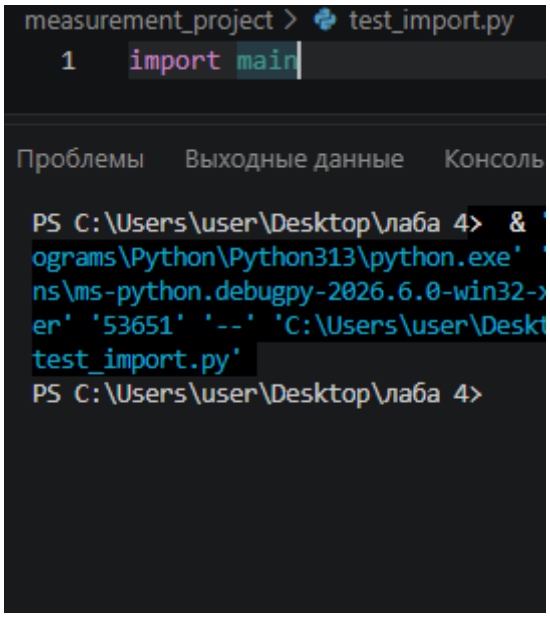
импорт функций в основной файл, проверка работы этих функций с тестовым списком

Задание 3

```python
from utils.statistics import calculate_average,find_min,find_max
def main():
    test=[4,34,532,325,654,-546,1134,452,123,75,2134,12]
    print(calculate_average(test))
    print(find_min(test))
    print(find_max(test))

if __name__ == "__main__":
    main()
```

Конструкция if __name__ == "__main__": main() - эта строка нужна для того, чтобы код выполнялся **только тогда, когда вы сами этого хотите** (при прямом запуске файла), и молчал, если этот файл просто подключили к другой программе как библиотеку.



При запуске файла test_import.py главным модулем для Python становится именно он. Когда внутри него происходит импорт import main, системная переменная __name__ внутри файла main.py принимает значение его имени — "main". Из-за этого проверка if __name__ == "__main__" возвращает ложь (False), и автоматический вызов функции main() успешно блокируется. Это доказывает, что наша точка входа изолирована и защищена.

Задание 4

```python
def read_numbers(file_path):
    numbers=[]
    with open(file_path, 'r', encoding='utf-8') as file:
        for line in file:
            numbers.append(float(line))
    return numbers
```

```
23
132
43124
532456
3212
43
-2345
54
34
13
```

Написание функции для прочтения чисел из ранее полученного файлика

Памятка:

with open(file_path, ‘r’, encoding=’utf-8’) as file:

“r” – режим read,только читать

encoding='utf-8' – кодировка. Без неё Windows может прочитать файл не в той кодировке где текст превратится в условный мусор

with ... as file: - открой файл, дай ему временное имя file, выполни весь код, который написан ниже с отступом, а как только этот код закончится (или если в программе произойдет ошибка) — автоматически закрой файл

```python
from utils.statistics import calculate_average,find_min,find_max
from utils.reader import read_numbers
def main():
    numbers=read_numbers('data/measurements.txt')
    print(numbers)
    print(calculate_average(numbers))
    print(find_min(numbers))
    print(find_max(numbers))

if __name__ == "__main__":
    main()
```

Импорт функции в основной файлик и замена им тестового списка

Задание 5

```python
from utils.statistics import calculate_average,find_min,find_max
from utils.reader import read_numbers
from pathlib import Path

def main():
    file_path=Path('data')/'measurements.txt'#соединение пути с резудьтатом (объект path)
    numbers=read_numbers(file_path)
    print(file_path)#путь
    print(file_path.exists())#существует ли файл
    log_dir=Path('logs')#создаёт папку виртуально что бы позже рабоать в ней
    log_dir.mkdir(exist_ok=True)#физиччески создаёт папку если папка есть то скип


    print(numbers)
    print(calculate_average(numbers))
    print(find_min(numbers))
    print(find_max(numbers))

if __name__ == "__main__":
    main()
```

В целом объект Path из модуля pathlib используется для обеспечения кроссплатформенности программы. Он автоматически адаптирует разделители путей (слэши) под конкретную операционную систему (Windows или Linux) и предоставляет встроенные методы (.exists() и .mkdir()) для безопасной работы с файловой структурой.

measurements.txt

Задание 6

Добавляем конструкцию try(ошибка): except(ошибка):

Что бы вместо большой ошибки выдавались заготовленное описание проблемы и можно было понять что не так грубо говоря защита

•С корректным файликом всё работает выдаёт всё что надо

```
<function read_numbers at 0x00000242ADFB9D00>
data\measurements.txt
False
файл не найден
```

•При переименовании исходного файла .txt на другое вместо огромной ошибки FileNotFoundError вылетает нами заданный текст при этой ошибки ‘файл не найден’ можно указать в принципе папку которой не существует

```
data\measurements.txt
True
не тот тип данных
```

•При вводе не тех данных то будет ошибка ValueError которую мы заменили на свой текст под текущую работу

Задание 7

```
def validate_numbers(numbers):
    if len(numbers)==0:
        raise ValueError("Список измерений пуст")
```

Создаём функцию с исключением что пустой список будет ошибкой ValueError а не FileNotFoundError

```
except ValueError as q:
    print(q)
```

Конструкция as q связывает пойманную ошибку с переменной q. Мы добавили её, чтобы программа могла заглянуть внутрь ошибки, прочитать её точный текст и сама определить, из-за чего именно произошёл сбой: из-за букв в файле или из-за пустого списка. Это позволило нам полностью заменить непонятные английские системные сообщения на красивый русский текст для пользователя. Но теперь если в списке будут не те данные будет выведено could not convert string to float: 'wd\n'

Задание 8

```
import logging
from utils.statistics import calculate_average,find_min,find_max
from utils.reader import read_numbers,validate_numbers
from pathlib import Path

def main():

    logging.basicConfig(
        filename="logs/app.log",
        level=logging.INFO,
        format="%(asctime)s %(levelname)s %(message)s",
        encoding="utf-8"
    )

    logging.info("Программа запущена.")

    file_path=Path('data')/'measurements.txt'#соединение пути с резудьтатом (объект path)
    print(file_path)#путь
    print(file_path.exists())#существует ли файл

    log_dir=Path('logs')#создаёт папку виртуально что бы позже рабоать в ней
    log_dir.mkdir(exist_ok=True)#физиччески создаёт папку если папка есть то скип

    try:

        numbers=read_numbers(file_path)
        logging.info(f"файл прочитан,количество измерений{len(numbers)}")
        validate_numbers(numbers)
        print(numbers)
        print(calculate_average(numbers))
        print(find_min(numbers))
        print(find_max(numbers))

    except FileNotFoundError:
        print('файл не найден')
        logging.info("ОШИБКА файл не найден")
    except ValueError as q:
        print(q)
        logging.info("ОШИБКА файл пустой/не тот тип данных")
    finally:
        logging.info("работа окончена")

if __name__ == "__main__":
    main()
```

```
2026-10-28 23:50:13,065 INFO Программа запущена.
2026-10-28 23:50:13,066 INFO файл прочитан,количество измерений10
2026-10-28 23:50:13,066 INFO работа окончена

```

Импортировали логи и настроили их по методичке сделали в программе текстовый блокнот app.log. Теперь при запуске код сам записывает туда точное время, когда он включился, сколько чисел прочитал из файла, какие ошибки выскочили и когда он выключился

Задание 9

config.json
```
{
  "input_file": "data/measurements.txt",
  "log_file": "logs/app.log"
}
```

Импортировали библиотеку json это инструмент, который позволяет программе понимать настройки, записанные во внешнем текстовом файле. Она берет текстовые данные из файла настроек, расшифровывает их структуру и превращает в удобную виртуальную картотеку. Благодаря этому программа получает возможность обращаться к настройкам по их именам и мгновенно доставать нужные пути к файлам данных или системным журналам. В нашей лабораторной работе это позволило полностью отделить настройки от логики кода, чтобы менять пути к файлам «на лету» без изменения самой программы

```
def main():
    with open('config.json','r',encoding='utf-8') as file:
        config=json.load(file)
    input_file=config['input_file']
    log_file=config['log_file']
    print(input_file)

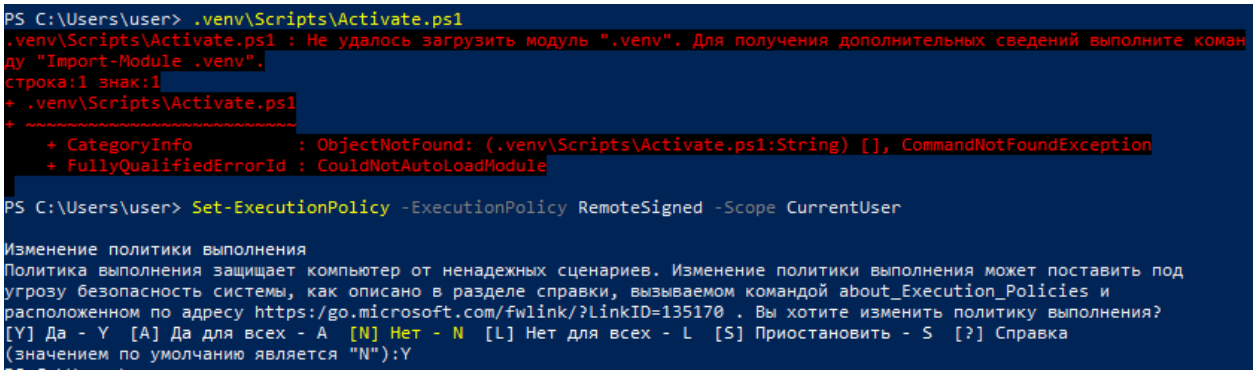
    logging.basicConfig(
        filename=log_file,
        level=logging.INFO,
        format="%(asctime)s %(levelname)s %(message)s",
        encoding="utf-8"
    )
```

Задание 10

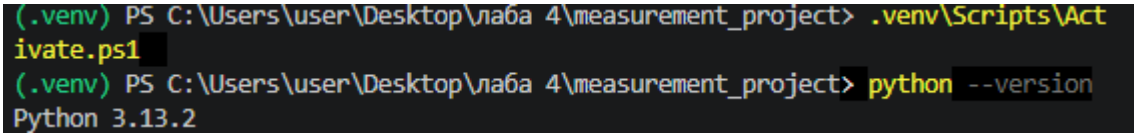
Создаём окружение в корне проекта python -m venv .venv

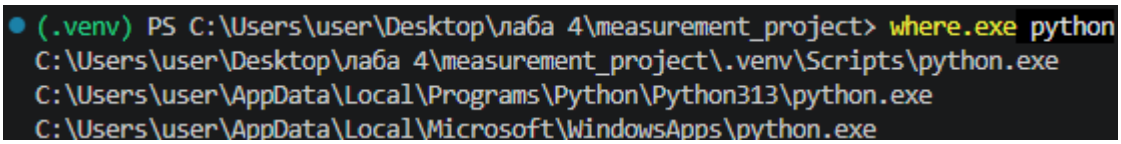
У каждого проекта — свой набор библиотек, и они не должны мешать друг другу. Без окружений: проекту A нужна библиотека X версии 1, проекту B — версии 2, а в общем Python место только для одной версии X. Обновили X для проекта B — проект A перестал работать. С окружениями: у каждого проекта свой каталог .venv со своим набором библиотек

При активации столкнулся с проблемой и там же решил её командой



Активируем и смотрим версию,где находится





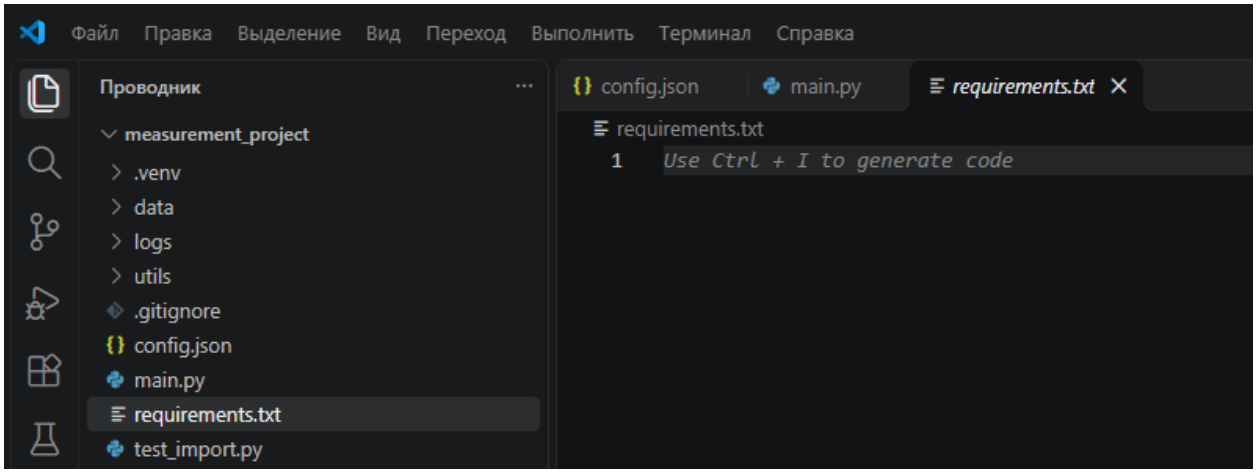
Проект переведён на использование локального интерпретатора из каталога .venv. Это гарантирует, что программа работает в полностью изолированной среде и защищена от конфликтов с глобальными библиотеками операционной системы

Задание 11

```
(.venv) PS C:\Users\user\Desktop\лаба 4\measurement_project> python -m pip lis
t
Package Version
------- -------
pip     24.3.1
```

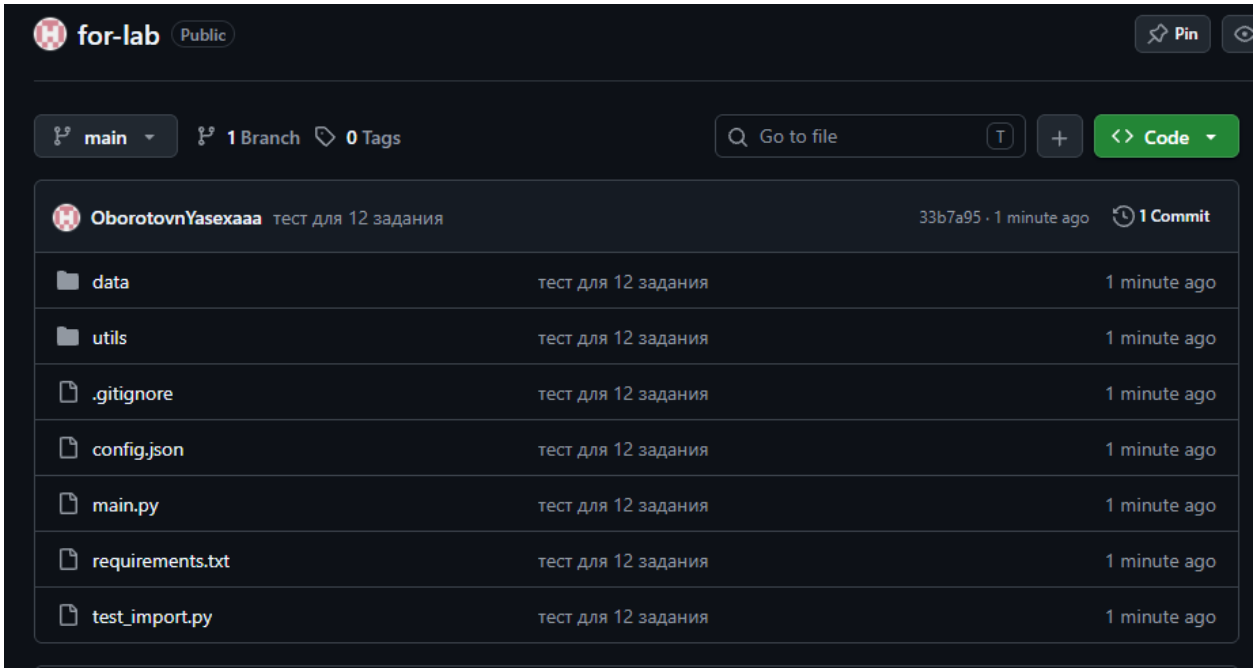
Терминал выводит короткую таблицу, так как наше окружение изолированное и абсолютно чистое.

```
(.venv) PS C:\Users\user\Desktop\лаба 4\measurement_project> python -m pip freeze > requirements.txt
(.venv) PS C:\Users\user\Desktop\лаба 4\measurement_project>
```



Сгенерированный файл требований requirements.txt оказался полностью пустым. Этот результат является абсолютно корректным, поскольку наша программа написана исключительно с использованием модулей стандартной библиотеки Python (json, logging, pathlib). Никаких сторонних библиотек из интернета в проект не скачивалось, поэтому фиксировать в зависимостях нечего.

Задание 12



Проверка работы .gitignore

Задание 13

ПРОВЕРКА ПРОЕКТА

```
2026-10-29 18:17:42,127 INFO Программа запущена.
2026-10-29 18:17:42,127 INFO файл прочитан,количество измерений10
2026-10-29 18:17:42,127 INFO работа окончена
```

```
data/measurements.txt
data\measurements.txt
True
[456.0, 2345.0, 2363.0, 96.0, -567.0, 23456.0, 90789.0, 3523.0, 567.0, 1234.0]
12426.2
-567.0
90789.0
```

При рабочем файле с теми данными которые подходят

```
2026-10-29 18:16:47,880 INFO Программа запущена.
2026-10-29 18:16:47,880 ERROR ОШИБКА файл не найден
2026-10-29 18:16:47,880 INFO работа окончена
```

```
data/measyayayayaurements.txt
data\measyayayayaurements.txt
False
файл не найден
```

При отсутствии файла или не правильном пути к файлу

```
2026-10-29 18:16:54,457 INFO Программа запущена.
2026-10-29 18:16:54,461 ERROR ОШИБКА файл пустой/не тот тип данных
2026-10-29 18:16:54,462 INFO работа окончена
```

```
data/measurements.txt
data\measurements.txt
True
could not convert string to float: 'факеуп3\n'
```

При ошибке если не тот тип данных или файл оказался пустым

Вывод

В ходе лабораторной работы мы получили полноценный многофайловый проект и освоили принципы его организации: код распределён между модулем чтения данных, модулем расчётов и точкой входа main.py, изменяемые параметры вынесены в config.json и меняются без правки кода. Корректность проверена тремя сценариями (корректный файл, отсутствующий файл, некорректные значения) — ожидаемые ошибки перехватываются try/except и фиксируются в журнале app.log с уровнями INFO и ERROR, программа не падает с трассировкой. Проект изолирован в виртуальном окружении .venv, зависимости описаны в requirements.txt, а служебные файлы исключены из репозитория правилами .gitignore.